

4교시: 버전 관리와 배포 협업

수업 목표

- 이미지 태그, 하위 호환, 배포 순서, rollback 가능성을 MSA 운영 문제로 설명한다.
- 서비스 경계, 네트워크, 설정, 로그, health 중 최소 두 가지 evidence로 정상 상태를 판단한다.
- 장애가 나면 재현, 관찰, 가설, 수정, 재확인 순서로 기록한다.

선행 지식

이미 알고 있어야 할 것	오늘 다시 연결할 것
Docker image/container	MSA의 서비스별 실행 단위
Docker Compose service	서비스 토폴로지와 의존성
port binding, service name DNS	내부 통신과 외부 진입점 구분
logs/status evidence	분산 로그와 장애 분석

50분 흐름

시간	활동	학습 초점	학생 산출
0-5분	문제 상황 제시	왜 단일 앱보다 운영이 어려워지는지 본다.	오늘의 질문 1개
5-15분	핵심 개념 설명	버전 관리와 배포 협업의 운영 의미를 정리한다.	용어 note
15-25분	구조/공식 기준 확인	service, dependency, health, logs를 공식 용어와 연결한다.	판단 표
25-40분	실습 또는 데모	Compose 상태와 로그로 실제 증거를 확인한다.	command evidence
40-47분	장애/비용/보안 점검	운영 위험과 복구 기준을 정리한다.	risk note
47-50분	다음 교시 연결	다음 service boundary로 확장한다.	handoff 문장

시작 상황

서비스가 하나일 때는 container가 켜졌는가 , port가 열렸는가 , HTTP 응답이 오는가 를 보면 대부분의 상태를 파악할 수 있다. MSA에서는 같은 질문을 서비스마다 반복해야 하고, 한 서비스의 정상 상태가 전체 사용자 경험의 정상 상태를 보장하지 않는다.

오늘의 주제인 버전 관리와 배포 협업 은 이 복잡도를 줄이기 위한 관찰 기준을 만든다. 인프라/DevOps 엔지니어는 내부 구현을 모두 알 필요는 없지만, 어떤 서비스가 어떤 주소로 누구에게 요청하고, 어떤 설정을 필요로 하며, 어디에서 실패 증거를 남기는지는 알아야 한다.

Generated Visual: 4교시 전용 인포그래픽

Week 3 Day 4 Lesson 4 - 버전 관리와 배포 협업

이미지 태그를 명확히 관리하고, 호환성을 확인하며, 안전한 순서로 배포하고, 문제가 생기면 빠르게 rollback 합니다.

1. 이미지 태그와 버전 관리

명확한 태그 규칙으로 추적성과 재현성을 확보합니다.



빌드
CI/CD



이미지 레지스트리

v1.0.0

v1.1.0

v1.1.1

latest

명시적 태그 사용 권장

- SemVer 권장 (MAJOR.MINOR.PATCH)
- 불변(Immutable) 태그 사용
- 빌드 시 버전/커밋 정보 포함
- 누가, 언제 배포했는지 추적 가능

2. latest 태그 위험

latest는 자동 변경되어 예기치 않은 장애를 유발할 수 있습니다.



latest

오늘은 v1.1.1

내일은 v1.2.0 ?

- 예기치 않은 버전 변경
- 재현 불가 / 추적 어려움
- 롤백 시 어떤 버전인지 불명확
- 프로덕션에서는 사용 금지

3. 하위 호환(Backward Compatibility)

API 변경 시 기존 클라이언트가 계속 동작해야 합니다.

호환 OK (권장)

- 선택적 필드 추가
- 기본값 유지
- 새로운 엔드포인트 추가
- 기존 응답 구조 유지

호환 깨짐 (주의/금지)

- 필수 필드 제거/이름 변경
- 응답 구조 변경
- 엔드포인트 삭제/경로 변경
- 데이터 타입 변경

계약(API Contract) 변경 시 버전 업(예: v2)

4. 배포 순서 전략

의존성 및 호환성을 고려한 안전한 배포 순서를 선택합니다.

전략 A. frontend → api

frontend가 api 변경을 요구하지 않는 경우



frontend
:80



api
:8080



database
:5432

- frontend 배포 (신버전)
- api 배포 (신버전)
- 검증 및 모니터링

전략 B. api → worker

api 변경이 worker에 영향을 주는 경우



api
:8080



worker
:8081



database
:5432

- api 배포 (신버전)
- worker 배포 (신버전)
- 검증 및 모니터링

5. 변경 영향 분석

바뀐 내용이 어떤 서비스에 영향을 주는지 파악합니다.



frontend



api



worker



database

변경 유형	예시	영향을 받는 대상
요청 스키마 추가	새 필드 추가(선택)	영향 없음 (하위 호환)
요청 스키마 변경	필드 삭제/이름 변경	frontend 영향 큼
응답 구조 변경	필드 삭제/타입 변경	frontend 영향 큼
엔드포인트 변경	경로 변경/삭제	frontend 영향 큼
내부 로직 변경	성능/버그 수정	worker, db 영향 가능
DB 스키마 변경	데이터형/값 변경	api, worker 영향 큼

6. rollback(롤백) 경로

문제 발생 시 빠르게 이전 버전으로 되돌립니다.

v1.0.0 (현재 운영)

배포

v1.1.0 (신버전)

문제 발생

v1.1.0 (실패)

롤백

v1.0.0 (이전 안정 버전)

롤백 체크리스트

- 이전 안정 버전 태그 확인
- 이미지/패키지드 재확인
- DB 변경이 있는 경우: 역마이그레이션 또는 호환 모드 확인
- 모니터링 및 가능 증상 여부 검증

7. release note & 커뮤니케이션

변경 내용을 문서화하고 팀과한 공유합니다.

release note 템플릿 예시

- 버전: v1.1.0
- 배포일: 2025-06-02
- 변경 요약(What): 주문 금액 필드 추가
- 변경 상세(Why/How): 할인 계산 정책 변경
- 호환성: 하위 호환 (기존 클라이언트 영향 없음)
- 영향 서비스: frontend, api
- 주의 사항: 없음
- 롤백 버전: v1.0.0

커뮤니케이션 채널

- 배포 공지 (Slack/이메일)
- 변경 내용 공유 (팀 회의)
- 배포 후 상태 공유 (모니터링 리포트)

핵심 요약



명시적 이미지 태그로 버전 관리



하위 호환을 지키고 계약을 관리



안전한 배포 순서로 변경 적용



변경 영향 분석으로 리스크 최소화



문제 발생 시 빠르게 rollback

기억하세요

- latest는 개발 환경에서만!
- 버전은 불변(Immutable)하게!
- 계약을 지키면 배포가 쉬워집니다!
- 문제는 빨리 발견하고, 빠르게 되돌립니다!

이 이미지는 Week 3 Day 4 4교시 버전 관리와 배포 협업 의 핵심 개념을 세션 전용으로 시각화한 자료다. 다른 교시와 같은 그림을 재사용하지 않고, 이 세션에서 확인할 운영 evidence와 판단 기준에 맞춰 읽는다.

Visual 1: 서비스 토폴로지 읽기

```
graph LR; User[User / Browser] --> Frontend[frontend  
external port 18083]; Frontend --> API[api  
internal port 8080]; API --> DB[(database  
PostgreSQL 5432)]; Worker[worker] --> API; Worker --> DB;
```

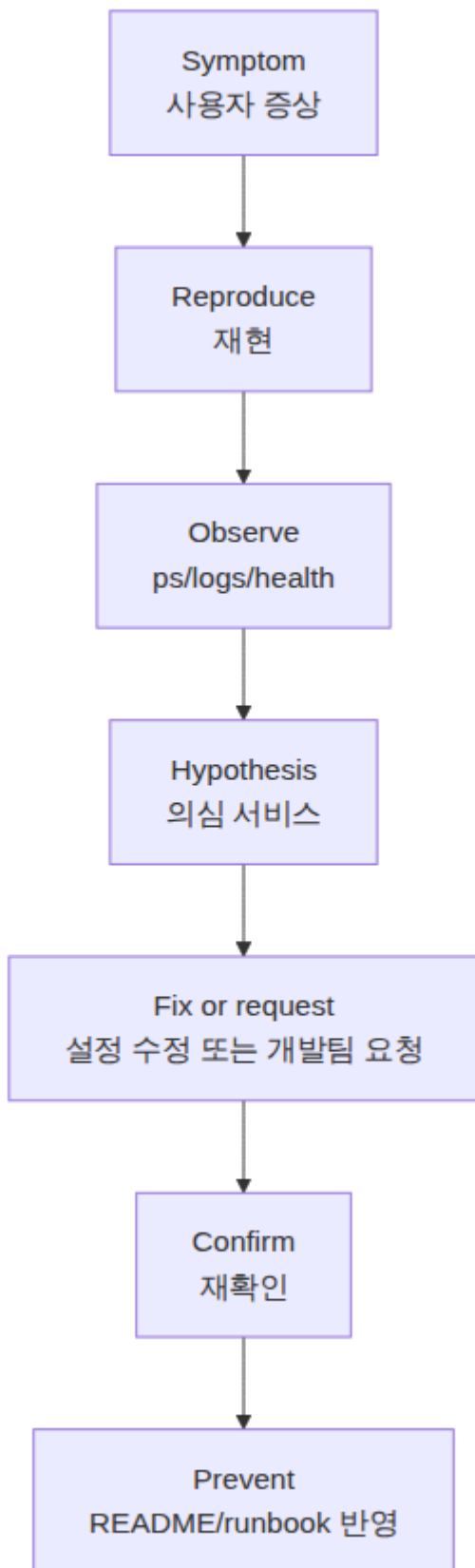
읽는 순서: 사용자는 frontend로 들어오고, frontend는 api로 요청을 넘긴다. api와 worker는 database에 의존한다. 이 그림에서 중요한 것은 박스 개수가 아니라, 장애가 어디에서 어디로 전파될 수 있는지 보는 것이다.

Visual 2: 운영 판단 표

확인 대상	정상 evidence	실패 증상	다음 행동
frontend	browser 접속, HTTP 200	화면은 뜨지만 API 데이터 없음	proxy/API URL 확인
api	/health, /api/status 응답	5xx, timeout, DB unavailable	env, DB host, logs 확인
worker	주기적 처리 로그	처리 정지, 반복 실패	dependency와 retry 주기 확인
database	healthcheck healthy	connection refused	volume, credential, readiness 확인

이 표는 기능 목록이 아니라 장애 분석 순서다. 운영자는 사용자가 본 증상에서 시작해 어느 서비스 evidence를 먼저 볼지 정해야 한다.

Visual 3: 장애 분석 흐름



이 흐름은 Week 1의 RCA와 같다. 달라진 점은 관찰 대상이 하나의 process가 아니라 여러 service의 조합이라는 점이다.

실습 기준

repository root에서 시작한다. Day 1 이후 모든 교시는 같은 실습 앱을 기준으로 진행한다.

```
cd week3/day1/labs/msa-demo
docker compose ps
docker compose logs --tail=40 api
docker compose logs --tail=40 worker
curl -s http://localhost:18083/api/status
```

기대 결과는 서비스 상태가 모두 `running` 또는 `healthy` 에 가깝고, `/api/status` 가 frontend를 통해 api 상태를 JSON으로 돌려주는 것이다. 실패하면 실패 자체를 evidence로 남긴다.

DevOps 원칙 연결

관점	오늘의 연결
비용 절감	로컬 Compose로 topology를 먼저 검증해 cloud 리소스 생성 전 오류를 줄인다.
개발/배포 효율	서비스별 실행 계약을 문서화해 개발팀과 운영팀의 왕복 질문을 줄인다.
관리 효율	health, logs, dependency map을 표준화해 장애 triage 시간을 줄인다.

흔한 오해

오해	바로잡기
MSA는 서비스를 많이 나누면 자동으로 좋아진다.	서비스 경계가 운영 비용보다 큰 이익을 줄 때 선택한다.
frontend가 보이면 전체 시스템이 정상이다.	API나 DB 장애가 숨어 있을 수 있으므로 service별 health를 봐야 한다.
depends_on이면 DB 준비까지 보장된다.	실행 순서와 readiness는 다르다. healthcheck와 retry가 필요하다.
로그를 보면 다 알 수 있다.	여러 서비스 로그를 연결할 request id/correlation id가 없으면 원인 추적이 느려진다.

Evidence Note 양식

```
# Week 3 Evidence - 버전 관리와 배포 협업

## Service checked
- Service:
- Command:
- Expected:
- Actual:

## Dependency
- Upstream:
- Downstream:
- Config/env used:

## Failure or risk
- Symptom:
- Observed evidence:
```

- Fix or request:
- Recheck result:

평가 기준

기준	통과 조건
개념 이해	버전 관리와 배포 협업을 서비스 경계/의존성/운영 증거 중 하나와 연결했다.
실행 증거	Compose 상태, 로그, HTTP 응답 중 하나 이상을 기록했다.
장애 분석	증상과 의심 서비스를 구분했다.
협업 기준	개발팀에 요청할 정보 또는 운영 문서에 남길 정보를 적었다.
다음 연결	Kubernetes에서 어떤 리소스로 이어질지 한 문장으로 설명했다.

공식/학술 근거 링크

- AWS Microservices on AWS, <https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices.html> - MSA를 여러 서비스와 API 기반 구조로 설명하는 공식 기준이다.
- Docker Compose Docs, <https://docs.docker.com/compose/> - 여러 service를 파일로 실행하고 관리하는 기준이다.
- Google SRE Book: Addressing Cascading Failures, <https://sre.google/sre-book/addressing-cascading-failures/> - 한 서비스 장애가 다른 서비스로 퍼질 수 있음을 설명하는 근거다.

다음 연결

다음 교시는 같은 실습 앱에서 다른 service boundary를 본다. 오늘 만든 evidence note는 서비스 목록, 의존성, health, logs를 누적하는 운영 문서가 된다.